

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ
ЕНЕРГЕТИКИ

РОЗРАХУНОК ГРАФІЧНА РОБОТА

З дисципліни:

«Візуалізація графічної та геометричної інформації»

Виконав:

студент групи ТР-52мп

Плесканко А.В.

Прийняв:

Демчишин А.А.

Київ 2025

1. Опис завдання

У рамках одинадцятого варіанту практичного завдання переді мною стояло завдання реалізувати масштабування текстури відносно довільної точки у просторі UV. Я мав взяти за основу код із контрольної роботи, де вже була реалізована модель поверхні Кассіні, текстурування, normal mapping, а також базовий інтерфейс і система рендерингу. Моєю метою було розширити цей функціонал, доповнивши його можливістю змінювати положення точки масштабування та сам коефіцієнт масштабу у режимі реального часу.

Я повинен був забезпечити повністю інтерактивну поведінку: точка масштабування мала переміщуватися завдяки клавішам клавіатури, а коефіцієнт масштабу мав збільшуватися або зменшуватися залежно від дій користувача. Також важливо було модернізувати фрагментний шейдер таким чином, щоб він коректно обчислював нові значення текстурних координат. Усі зміни повинні були вписатися у наявну архітектуру проєкту, не порушуючи роботу освітлення, текстур і нормалей.

У підсумку я мав отримати WebGL-додаток, у якому текстура на моделі плавно змінюється відповідно до вибраного центру масштабування, а користувач може вільно досліджувати ефект шляхом переміщення камери та керування текстурою. Після реалізації я також підготував цей структурований звіт і розмістив вихідний код у гілці CGW свого GitHub-репозиторію.

2. Теоретичні відомості

Перш ніж переходити до реалізації, я опрацював питання, що стосуються трансформацій текстурних координат. Під час рендерингу WebGL використовує UV-координати для вибірки точок на текстурному зображенні, і будь-яке перетворення цих координат безпосередньо впливає на вигляд текстури на моделі. Я усвідомлював, що масштабування текстури є афінним перетворенням, яке в UV-просторі проявляється як зміна відстані між вибірками текстури.

Оскільки за умовою варіанта я мав опрацьовувати масштабування не відносно початку координат, а відносно довільної точки, мені було необхідно правильно застосувати відповідну формулу. Я працював із перетворенням такого вигляду:

$$u', v' = P + (UV - P) \times S$$

де P є точкою масштабування, а S коефіцієнтом масштабу. Для мене було важливо розуміти, що спочатку координати повинні бути приведені до системи, де обрана точка стає центром, далі має відбутися масштабування, а після цього координати повертаються назад у початкову систему.

У процесі роботи я також звернув увагу на те, як у шейдері виконується normal mapping. Оскільки я використовую нормалі з текстурної карти, необхідно було упевнитись, що TBN-матриця побудована коректно. Через це у вершинному шейдері я ортогоналізую вектори tangent і normal, щоб уникнути можливих спотворень при освітленні. Далі нормалі з normal map переносяться у простір камери і беруть участь у розрахунку моделі освітлення Фонга. Ця модель забезпечує амбієнтну, дифузну та дзеркальну складові освітлення, що у поєднанні з картами дифузної та спекулярної складових дає якісний візуальний результат.

3. Реалізація

Почав я із фрагментного шейдера, оскільки саме там відбувається вибірка текстурних даних. Я додав дві нові uniform-змінні: точку масштабування та коефіцієнт масштабу. Після цього я реалізував обчислення нових координат відповідно до теоретичної формули. Конкретно це виглядає так:

$$\text{scaledUV} = \text{u_pivotPoint} + (\text{vUV} - \text{u_pivotPoint}) \times \text{u_textureScale}$$

Отримане значення я використовую для вибірки значень із дифузної, спекулярної та карти нормалей. Таким чином я можу коректно зміщувати текстуру відносно будь-якої точки, яку користувач вибере у процесі взаємодії.

Після цього я перейшов до JavaScript частини програми. Я реалізував обробник клавіатурних подій, завдяки якому користувач може рухати точку масштабування у межах від 0 до 1 по кожній осі UV. Для цього я використав клавіші A, D, W та S. Масштаб текстури я пов'язав із клавішами Q та E. Для зручності передбачив можливість миттєвого скидання параметрів кнопкою R. Я також подбав про те, щоб значення оновлювалися в інтерфейсі і щоб це не впливало на FPS.

Далі я забезпечив передачу параметрів у фрагментний шейдер у функції draw. Поряд із іншими uniform-ами я додав відправлення pivotU, pivotV та textureScale. Завдяки цьому шейдер у кожному кадрі отримує актуальні дані, що дозволяє здійснювати плавну та безперервну взаємодію.

4. Інструкція користувача

Після запуску програми на екрані з'являється тривимірний модель поверхні Кассіні із застосованими картами дифузної текстури, спекулярної текстури та normal map.

CGW: Texture Coordinate Scaling (Variant 11)

Drag your mouse on the surface to rotate it.

U Segments: 40 
V Segments: 40 

Texture Scaling Controls:
Pivot Point: U: 0.50, V: 0.50
Scale Factor: 1.00

W/S - Move pivot along V axis
A/D - Move pivot along U axis
Q/E - Decrease/Increase scale
R - Reset to defaults



Рисунок 1. Початковий вигляд моделі з накладеними текстурами.

Користувач має можливість змінювати масштаб текстури (Texture Scale) у реальному часі. Натисканням клавіш Q (зменшити масштаб / Zoom In) та E (збільшити масштаб / Zoom Out). При збільшенні значення масштабу (наприклад, до 3.0) текстура стає дрібнішою, а її повторюваність на поверхні збільшується.

CGW: Texture Coordinate Scaling (Variant 11)

Drag your mouse on the surface to rotate it.

U Segments: 40

V Segments: 40

Texture Scaling Controls:
Pivot Point: U: 0.50, V: 0.50
Scale Factor: 3.70

W/S - Move pivot along V axis

A/D - Move pivot along U axis

Q/E - Decrease/Increase scale

R - Reset to defaults

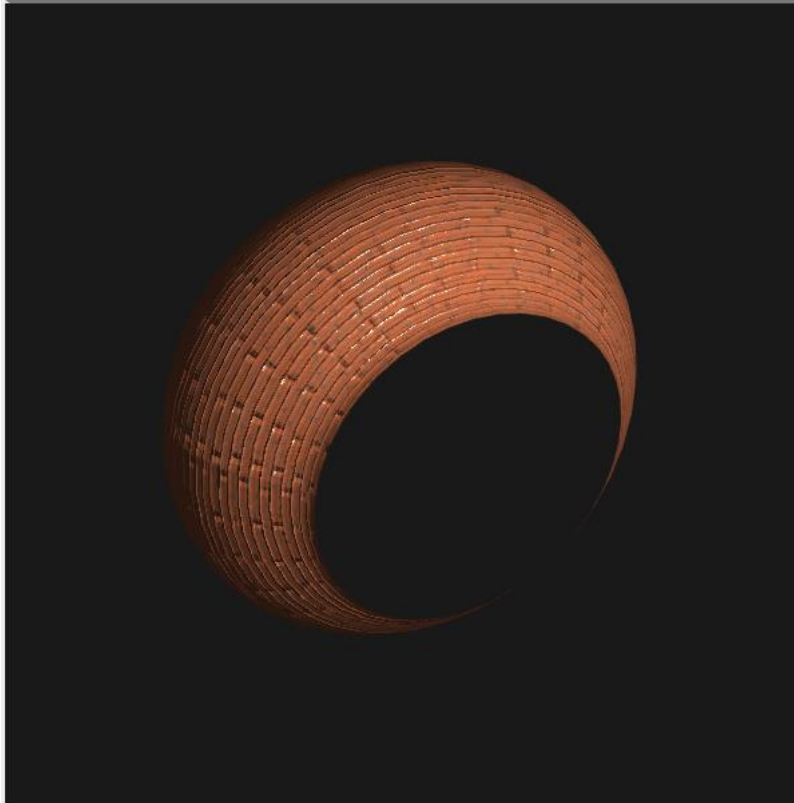


Рисунок 2. Результат масштабування текстури.

Ключовою особливістю програми є можливість масштабування відносно довільної точки (Pivot Point). За замовчуванням центр знаходиться посередині текстурних координат (0.5, 0.5). Для переміщення точки масштабування необхідно використовувати клавіатуру:

W / S — переміщення центру вгору або вниз (по осі V).

A / D — переміщення центру вліво або вправо (по осі U).

При зміні центру візуальний ефект "стягування" текстури зміщується у відповідний бік, а на інформаційній панелі оновлюються координати точки Pivot.

CGW: Texture Coordinate Scaling (Variant 11)

Drag your mouse on the surface to rotate it.

U Segments: 40

V Segments: 40

Texture Scaling Controls:

Pivot Point: U: 1.00, V: 0.96

Scale Factor: 5.00

W/S - Move pivot along V axis

A/D - Move pivot along U axis

Q/E - Decrease/Increase scale

R - Reset to defaults

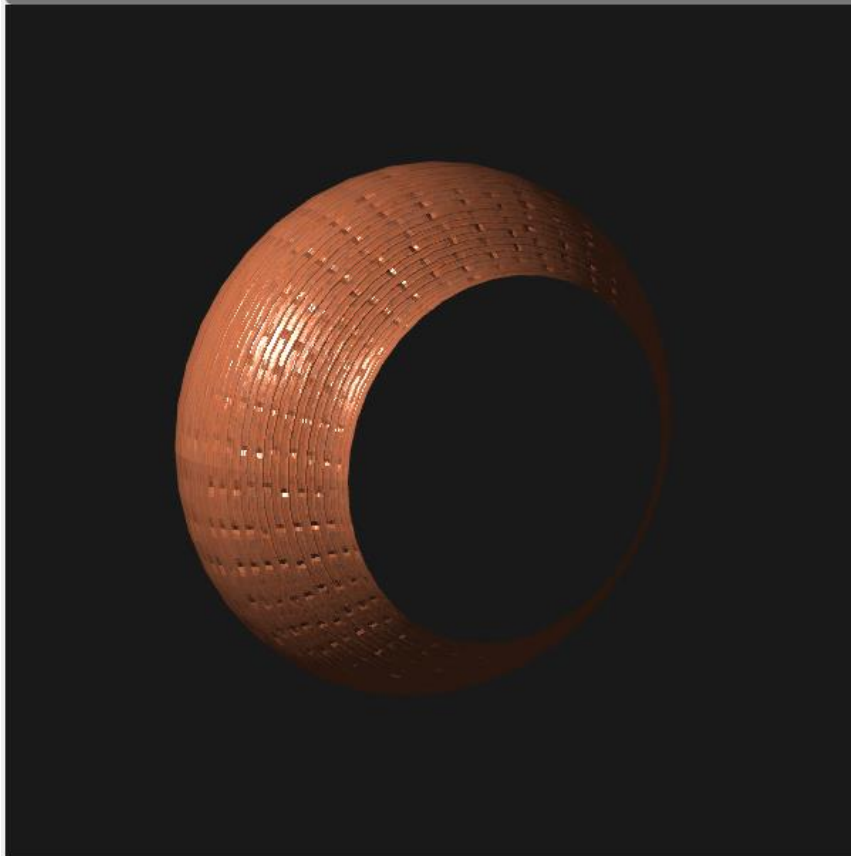


Рисунок 3. Масштабування текстури зі зміщеним центром (Pivot Point).

Для повернення всіх параметрів до початкових значень (центр у 0.5, масштаб 1.0) передбачено клавішу R.

5. Фрагмент коду

5.1. Fragment Shader (Texture Scaling Implementation)

```
const fragmentShaderSource = `

precision mediump float;

// ... (other uniforms)

uniform sampler2D diffuseTexture;

uniform sampler2D specularTexture;

uniform sampler2D normalTexture;


// CGW: Texture scaling uniforms (Variant 11)

uniform vec2 u_pivotPoint;

uniform float u_textureScale;


void main(void) {

    // Sample textures

    // CGW: Scaling formula: uv_new = pivot + (uv_old - pivot) * scale

    vec2 scaledUV = u_pivotPoint + (vUV - u_pivotPoint) * u_textureScale;


    vec4 diffuseTex = texture2D(diffuseTexture, scaledUV);

    vec4 specularTex = texture2D(specularTexture, scaledUV);
```

```
vec3 normalMap = texture2D(normalTexture, scaledUV).rgb;
```

```
// ... (lighting calculations continue) ...
```

```
}
```

```
`;
```

5.2. Keyboard Event Handling (Interactivity)

```
function setupKeyboardControls() {
```

```
    document.addEventListener('keydown', function (event) {
```

```
        let needsUpdate = false;
```

```
        switch (event.key.toLowerCase()) {
```

```
            case 'a': // Move pivot Left
```

```
                pivotU = Math.max(0, pivotU - pivotStep);
```

```
                needsUpdate = true;
```

```
                break;
```

```
            case 'd': // Move pivot Right
```

```
                pivotU = Math.min(1, pivotU + pivotStep);
```

```
                needsUpdate = true;
```

```
                break;
```

```
            case 'w': // Move pivot Up
```

```
    pivotV = Math.min(1, pivotV + pivotStep);

    needsUpdate = true;

    break;

case 's': // Move pivot Down

    pivotV = Math.max(0, pivotV - pivotStep);

    needsUpdate = true;

    break;

case 'q': // Zoom In

    textureScale = Math.max(0.1, textureScale - scaleStep);

    needsUpdate = true;

    break;

case 'e': // Zoom Out

    textureScale = Math.min(5.0, textureScale + scaleStep);

    needsUpdate = true;

    break;

case 'r': // Reset to defaults

    pivotU = 0.5;

    pivotV = 0.5;

    textureScale = 1.0;

    needsUpdate = true;
```

```

        break;
    }

    if (needsUpdate) {
        updateScalingUI();
        event.preventDefault();
    }
});
}

```

5.3. Draw Function (Uniforms Update)

```

function draw() {
    shProgram.Use();

    // CGW: Pass texture scaling uniforms to shader
    gl.uniform2fv(shProgram.iPivotPoint, [pivotU, pivotV]);
    gl.uniform1f(shProgram.iTextureScale, textureScale);

    if (diffuseTexture && specularTexture && normalTexture) {
        // ... (binding textures) ...
    } else {
        return;
    }
}

```

}

// ... (drawing elements) ...

}